School of Electrical & Computer Engineering

Number Theory & Cryptography

Assignment 1

Due by: March 24th 2026
Course Instructor: Prof. Georgios Karystinos
Assignment Supervisor: Ioannis-Leonidas Steiakakis

Group 12

Name	AM
Georgios-Angelos Gravalos	2021030001
Rafaela-Aikaterina Kerimi	2021030007

Contents

Task 1 - Preliminaries **2**

Task 2 - Euclidean Algorithm **3**

Task 3 - Hill Cipher **5**
3.a 5
3.b 5

Task 4 - A Decentralized Authentication Protocol **6**
4.a 7
4.b 8
4.c 8

Task 5 - A User Authentication Protocol **10**
5.a 10
5.b 10
5.c 10
5.d 10
5.e 11
5.f 11
5.g 11
5.h 11
5.i 12
5.j 12

Task 1 - Preliminaries

(Task 1.a) - 0%

- What is the life duration of a human (assume 100 years) in nanoseconds expressed in powers of 2? $\approx 2^{61.4}$ nanoseconds
- What is 10 times the above duration (1000 years) in nanoseconds expressed in powers of 2? $\approx 2^{64.7}$ nanoseconds
- What is the age of the universe (13.8 billion years) in nanoseconds expressed in powers of 2? $\approx 2^{88.5}$ nanoseconds

Before we get to the other tasks, as asked in the prompt, we will now define the follwoing operators:

- $\mathcal{T}_n \triangleq \{a \in \mathbb{Z}_n \mid 0, 1, .., n\}$, as also mentioned in (4.5) in our notes.
- $s \xleftarrow{\$} S$ denotes sampling an element s, uniformly at random, from set S.

To sample from a proper subset of $\mathbb{Z}_n$, first we sample from $\mathbb{Z}_n$, and then do rejection sampling.

Task 2 - Euclidean Algorithm

(2a) Euclidean Algorithm - 5% (`task2.py`)

We will now implement the accelerated Euclidean algorithm, standard and extended.
First of all, we ought to begin by mentioning the logic of the Euclidean algorithm.

$$\text{If } a = qb + r, \text{ with } a \neq 0 \text{ or } b \neq 0, \text{ then } (a, b) = (b, r). \tag{1.28}$$

At each step, we perform $r_{k-1} = q_k r_k + r_{k+1}$, with $0 \leq r_{k+1} < r_k$. Therefore, the remainder is always positive, and smaller than the divisor.

- **Standard accelerated version** The accelerated version speeds this up, by choosing the smallest absolute value, between the positive remainder and the negative remainder (therefore increasing or decreasing the quotient by ± 1). This standard version only returns the GCD of the two numbers given as input.
  The reasoning behind our implementation of the accelerated version is as follows: This is the v3 version of the non-accelerated euclidean algorithm, from our notes.

```
procedure GCD(a,b), with a>=b:
    repeat
        divide a by b: a = qb + r
        a <- b
        b <- r
    until b = 0
return a
```

  If we slightly revise this to allow both negative and positive remainders. This means that at each step, the next step will either be $a = qb + r$ again, or $a = (q + 1)b + (r - b)$

```
procedure GCD(a,b), with a>=b:
    repeat
        divide a by b: a = qb + r
        if |r-b| < |r|:
            q <- q+1
            r <- r-b
        a <- b
        b <- r
    until b = 0
return a
```

- **Extended accelerated version**

To avoid working backwards in order to build the linear combination of the two numbers, we use the Extended version of the accelerated Euclidean algorithm, which returns two coefficients x,y along with the gcd, such that $ax + by = gcd$. The methodology exists in our notes.
Since in every step $a = qb + r \iff r = a - qb$, each remainder is written as the linear combination of the numbers a,b.
We start with $r_{-1} = a, r_0 = b$, since $a = 1 \cdot a + 0 \cdot b$ and $b = 0 \cdot a + 1 \cdot b$. Therefore the first two coefficients

are $(x, y) = (1, 0)$ and $(x, y) = (0, 1)$.
This expands further:
.....
$r_{k-2} = a \cdot x_{k-2} + b \cdot y_{k-2}$
$r_{k-1} = a \cdot x_{k-1} + b \cdot y_{k-1}$
$r_k = a \cdot x_k + b \cdot y_k$

And based on the general formula for every step : $r = a - qb$, we have:
$r_k = r_{k-2} - q_k r_{k-1} = (a \cdot x_{k-2} + b \cdot y_{k-2}) - q_k(a \cdot x_{k-1} + b \cdot y_{k-1}) = a(x_{k-2} - q_k x_{k-1}) + b(y_{k-2} - q_k y_{k-1})$,
thus giving us:
$x_k = x_{k-2} - q_k x_{k-1}$
$y_k = y_{k-2} - q_k y_{k-1}$

So when the algorithm finally reaches $r_k = (a, b)$, then $r_k = ax_k + by_k$.

Task 3 - Hill Cipher

(3a) - 0%

In this task, we implemented the Hill Cipher while initially ignoring the constraint on the encryption keys. The implementation consists of the `keygen`, `encrypt`, and `decrypt` functions.

- **Key Generation (`keygen`)**: The encryption key is a quadruple $a, b, c, d \in \mathbb{Z}_M$. In this initial version, these four values are sampled uniformly at random from $\mathbb{Z}_M$ using `randbelow` without ensuring $(ad - bc, M) = 1$.

- **Encryption (`encrypt`)**: Following the lecture notes, two messages m_1, m_2 are encrypted into r_1, r_2 using the following linear congruences:

$$\begin{cases} r_1 = r_M[am_1 + bm_2] \\ r_2 = r_M[cm_1 + dm_2] \end{cases} \tag{6.8}$$

- **Decryption (`decrypt`)**: Decryption involves solving the system of linear congruences in $\mathbb{Z}_M$:

$$\begin{cases} am_1 + bm_2 \equiv r_1 \pmod{M} \\ cm_1 + dm_2 \equiv r_2 \pmod{M} \end{cases} \tag{6.9}$$

Does it work correctly?
No, the system does not work correctly. According to Theorem 2.32, the system above has a unique solution in $\mathbb{Z}_M$ if and only if $(ad - bc, M) = 1$. By ignoring this constraint, the determinant $(ad - bc)$ may not have a modular inverse t in $\mathbb{Z}_M$. Per Definition 2.26, this unique inverse exists if and only if the determinant and M are coprime. If $(ad - bc, M) > 1$, the decryption process will fail to produce a unique result, meaning the original messages cannot be recovered.

(3b) - 10%

For Task 3.b, we refined the implementation to ensure the key is valid for decryption while maintaining maximum randomness.

According to Theorem 2.32, for the Hill Cipher system to have a unique solution in $\mathbb{Z}_M$, the condition $(ad - bc, M) = 1$ must be satisfied. This ensures that $(ad - bc)$ has a unique modular inverse $t \in \mathbb{Z}_M$ (the solution to $(ad - bc)x \equiv 1 \pmod{M}$). The decrypted messages are then calculated as:

$$\hat{m}_1 = r_M[(dr_1 - br_2)t], \quad \hat{m}_2 = r_M[(ar_2 - cr_1)t]$$

To satisfy $(ad - bc, M) = 1$ while keeping the parameters as random as possible, we follow the methodology outlined in the assignment prompt:

1. **Sample** $k \leftarrow \mathcal{T}_M$: We first sample k such that $(k, M) = 1$. By Lemma 1.28, this ensures the resulting determinant $r_M[ad - bc]$ is a valid element of $\mathcal{T}_M$. We use rejection sampling to sample from $\mathcal{T}_M$.

2. **Random Selection**: We choose randomly each time, whether to sample the pair $[a, d]$ or $[b, c]$ first to avoid determinism.

3. **Solving for Remaining Parameters**:

   - If $[a, d]$ are sampled from $\mathbb{Z}_M$, we must find b, c such that $ad - bc \equiv k \pmod{M}$. This is rearranged into a linear congruence of the form $bc \equiv ad - k \pmod{M}$.

- Here,we choose again randomly whether to sample b or c.
- Let's say b is chosen. According to Theorem 2.21, this congruence is solvable for c if and only if $(b, M) \mid (ad - k)$. Our code samples b and checks this solvability condition before using the `solve_linear_congruence` function to find c.

The same procedure is followed for either starting pair chosen to be sampled and for either of the remaining parameters later.

`solve_linear_congruence(a, b, n)`:

According to Theorem 2.21 (1), the linear congruence $ax \equiv b \pmod{n}$ has a solution if and only if the greatest common divisor $d = (a, n)$ divides b. The function uses `extended_euclidean(a, n)` to find d and verifies this condition via `if b % gcd != 0`.

To simplify the calculation, the function applies the method from Subsection 2.3.2. We define:

$$a' = \frac{a}{(a,n)}, \quad b' = \frac{b}{(a,n)}, \quad n' = \frac{n}{(a,n)}$$

This transforms the original problem into the equivalent linear congruence $a'x \equiv b' \pmod{n'}$, where $(a', n') = 1$ is guaranteed.

Since $(a', n') = 1$, Corollary 2.25 and Definition 2.26 state that a' has a unique modular inverse t in $\mathbb{Z}_{n'}$. This inverse is obtained from the x coefficient of the `extended_euclidean` algorithm, which satisfies $ax + ny = (a, n)$.

The base solution x_0 is then computed by multiplying the inverse by the reduced target value:

$$x_0 = r_{n'}[t \cdot b']$$

While the original congruence has (a, n) mutually incongruent solutions modulo n per Corollary 2.22, this function returns the unique solution in $\mathbb{Z}_{n'}$ as required for cryptographic primitives like the Hill Cipher or the Chinese Remainder Theorem.

The `encrypt` and `decrypt` functions stayed the exact same.

This constrained key generation ensures that original messages are always recovered perfectly while the key space remains as unpredictable as possible.

Task 4 - A Decentralized Authentication Protocol

We are decentralizing the process of authenticating a party, by having many authentication servers, and every server knows a small part of the secret x. Therefore, we are given:

- Modulus n, in its canonical form $n = p_1^{k_1} p_2^{k_2} ... p_l^{k_l}$
- Secret $x \in \mathbb{Z}_n$
- l authentication servers, eacch server i holds a part of the modulus n, and we denote this part as $n_i = p_i^{k_i}$. This means that $n = n_1 \cdot n_2 \cdot ... \cdot n_l$, where $n_i = p_i^{k_i}$.
- The value known by each server is $r_{n_i}[n_{i-1}^n \cdot x]$, where $n_i = p_i^{k_i}$ and $n_0 = n_l$, to ensure that the first server has a valid value to use in its calculation.
  So for example, the first server holds $r_{n_1}[n_0^n \cdot x] = r_{n_1}[n_l^n \cdot x]$. The second server holds $r_{n_2}[n_1^n \cdot x]$, ..., up to server l, which holds $r_{n_l}[n_{l-1}^n \cdot x]$

Now, when a candidate party wants to be authenticated, it sends through the secure channel, to the authentication server i, the value $r_{n_i}[n_{i-1}^n \cdot x']$. The value x' is the secret x that the candidate party claims to hold.
If truly $x' = x$, then $n_{i-1}^n \cdot x' \equiv n_{i-1}^n \cdot x \pmod{n_i}$, for all i.
If every server finds the received value equal to its own, then the party is accepted, and rejected otherwise.

(4a) - 5%

Describe the possible values of x that each server can infer. Is the set of these values a complete set of residues (mod n_i)?

Each server i, holds $s_i = r_{n_i}[n_{i-1}^n \cdot x]$, where $r_{n_i} \in Z_{n_i}$ and therefore $s_i \in Z_{n_i}$.
According to our notes:

$$a \equiv r_n[a] \pmod{n} \qquad \text{(Property 7a, from 2.3)}$$

Which means that

$$n_{i-1}^n \cdot x \equiv s_i \pmod{n_i}$$

This is a linear congruence, which is solvable iff $(n_{i-1}^n, n_i) \mid s_i$.
In our case, $n_i = p_i^{k_i}$ and $n_{i-1} = p_{i-1}^{k_{i-1}}$ are different primes, that belong to the same canonical form. Therefore $(n_i, n_{i-1}) = 1$. Since n_i and n_{i-1} are co-prime, then n_i and a power of n_{i-1} are also co-prime.
This effectively means that the congruence isn't just solvable, but also has a unique solution in $\mathbb{Z}_{n_i}$.
So far, we know that EACH server i can calculate a single value x_0, such that $x \equiv x_0$ (mod n_i). But, Z_{n_i} is much smaller than Z_n, which is where the secret x belongs.
Each server doesn't know the entire x, it only knows an x_0, which leaves the same remainder as x when divided by n_i.
Converting the congruence to a diophantine, we have:

$$n_{i-1}^n x_0 - n_i y_0 = s_i$$

According to our notes, when we have a solution x_0 (we're not interested in y_0 for this), then all the solutions are:

$$x = x_0 + \frac{-n_i}{(n_{i-1}^n, -n_i)} t, \quad t \in \mathbb{Z} \qquad \text{(Theorem 1.31, 2)}$$

We already know that $(n_i, n_{i-1}^n) = 1$, and t can be any integer, therefore it's convenient for us to say:

$$x = x_0 + n_i \cdot t, \quad t \in \mathbb{Z} = \{..., x_0 - 2n_i, x_0 - n_i, x_0, x_0 + n_i, x_0 + 2n_i, ..\}$$

Since the secret $x \in \mathbb{Z}_n$, then $0 \leq x < n$.
Therefore, the set we're looking for is not the above, since this extends in $\mathbb{Z}$, not $\mathbb{Z}_n$.
The set we're looking for is the intersection of the above set, with $\mathbb{Z}_n$, where the secret x lives.

$$\{x_0 + tn_i \mid t \in \mathbb{Z}\} \cap \mathbb{Z}_n = \{x_0, x_0 + n_i, x_0 + 2n_i, .., x_0 + (\frac{n}{n_i} - 1)n_i\}$$

Two small things here to highlight:

- Since the last element of this set is $x_0 + (\frac{n}{n_i} - 1) \cdot n_i = x_0 + n - n_i$, and $x_0 \in \mathbb{Z}_{n_i}$, then this surely keeps our range of solutions within $0 \leq x < n$.
- Solutions smaller than x_0 are not kept after the intersection with $\mathbb{Z}_n$, since $x_0 \in Z_{n_i}$. This means that if we were to include, for example, $x_0 - n_i$, then this would no longer belong in $\mathbb{Z}_n$.

These are the solutions of x that the server can infer.

As for whether this set of solutions is a complete set of residues (mod n_i), it is not.
A complete set of residues (mod n_i) means that every integer inside the set is congruent to exactly one unique number in $\mathbb{Z}_{n_i}$, modulo n_i.
However, all of the elements in the set of solutions $\{x_0, x_0 + n_i, x_0 + 2n_i, ..., x_0 + (\frac{n}{n_i} - 1) \cdot n_i\}$ are congruent to the same remainder, x_0, and could therefore not be a complete set of residues (mod n_i).

(4b) - 10%

We believe that the code implementation is sufficiently documented through the comments in the code, however we will shortly present our implementation here in the report.

- We first generate (unique) 256-bit primes, and we insert them in a list `p_list`
- We then generate uniformly the powers $k \in \mathbb{Z}_{10}$, raise each prime in `n_list` to the power of each k, and then each $p_i^{k_i}$ is added to a list `n_list`
- We produce n as $n = p_1^{k1} \cdot p_2^{k_2} \cdot ... \cdot p_l^{k_l} = \Pi_{i=1}^{l} p_i^{k_i}$, and then the secret x is selected randomly in the range $\mathbb{Z}_n$
- Next, we store each server's secret $r_{n_i}[n_{i-1}^n \cdot x]$ in a list `server_content`. n_i and n_{i-1} for each server are taken from the list elements `n_list[i]` and `n_list[i-1]`, respectively. Each server's secret is stored in a list `coefficient_list` so we can have them ready for use if needed.
- To perform efficient modular exponentiation, we have written the function `efficient_pow()` in `utils.py`, which implements the binary representation of the exponent, to do fast computation of exponentiations with large powers, modulo a number. **(Theorem 2.11 from our notes, as well as example 2.12 which showcases the algorithm)**
- It is also important to note here, that in an attempt to speed up the algorithm, we utilize the **property (c) from Proposition 1.6** within the notes : $r_n[ab] = r_n[r_n[a]b] = r_n[r_n[a]r_n[b]]$, so as to reduce the secret x mod n_i during each server's secret calculation process, as well as during the authentication process.
  This makes $r_{n_i}[n_{i-1}^n \cdot x] = r_{n_i}[n_{i-1}^n \cdot r_{n_i}[x]]$
- Finally, the function `auth_party` performs the authentication of a party/client.
  For each server, the client calculates the secret they think the server holds, based on the secret x' that the client claims is $x' = x$. If the client is telling the truth, and $x' = x$, then each server's part $r_{n_i}[n_{i-1}^n \cdot x]$ will match what was sent by the client $r_{n_i}[n_{i-1}^n \cdot x']$, and the client will pass the authentication.

We have also included two tests in the code, which will run automatically with the script. In the first test, the client presents the correct secret $x' = x$ and is verified. In the second test, the client presents a secret $x' \neq x$, and is rejected at the first server they encounter.

(4c) - 15%

Is the protocol secure if every server is untrustworthy? If yes, prove it theoretically. If no, design theoretically and implement in code (using your implementation from 4b) an attacker

that extracts x.

Consider a brute force attack as infeasible. If every server is untrustworthy, then an attacker can find the secret x.

Since every server has stored $r_{n_i}[n_{i-1}^n \cdot x]$, and $(n_{i-1}, n_i) = 1$, an attacker essentially collects one linear congruence from each server:

$$n_{i-1}^n \cdot x \equiv s_i \pmod{n_i}$$

which has a unique solution $x_i \in \mathbb{Z}_{n_i}$.

This means that when the attacker has intercepted all the servers, they have collected a system of linear congruences:

$$\begin{cases} x \equiv x_1 \pmod{n_1} \\ x \equiv x_2 \pmod{n_2} \\ \vdots \\ x \equiv x_l \pmod{n_l} \end{cases} \qquad \text{(2.14 from our notes)}$$

This brings us to the Chinese Remainder Theorem, as mentioned in our notes, in **Theorem 2.27**: Since $n = n_1 \cdot n_2 \cdot ... \cdot n_l = p_1^{k_1} \cdot p_2^{k_2} \cdot ... \cdot p_l^{k_l}$, and for any two $n_i, n_j : (n_i, n_j) = 1$ (since each composite part of n is a distinct prime), then the above system is solvable and has a unique solution in $\mathbb{Z}_n$.

As for the code implementation:

Since every server is untrustworthy, the attacker has access to the lists we mentioned above, in 4b : `n_list,coeff_list, server_content`.

The attacker iterates through every server and extracts the solution x_i for every $s_i = r_{n_i}[n_{i-1}^n \cdot x]$ that the server holds, by solving the linear congruence using the extended euclidean algorithm to find the modular inverses for each server. The function `solve_linear_congruence(a,b,n)` is guaranteed to return a single solution, since we have proved above that each linear congruence has a unique solution in each $\mathbb{Z}_{n_i}$.

All x_i values are added to the list `reconstructed_xi`. Then, this list is used as explained in the proof of the Chinese Remainder Theorem (2.27), to compute the sum $x = \sum(x_i \cdot M_i \cdot y_i)$(mod n), where $M_i = n/n_i$, and y_i is the modular inverse of $M_i \pmod{n_i}$.

We ran the example where the client provides $x' = x$, and the result is that the attacker achieves to successfully reconstruct the secret when every server is untrustworthy. For the protocol to remain secure, at least one server has to be trustworthy.

Task 5 - A User Authentication Protocol

(5a) - 4%

Can an eavesdropper extract x? Justify your answer.
No. An eavesdropper observes n, y, b, c, and d. In the case $c = 0$, $d = a$ is simply a random commitment. In the case $c = 1$, $d = ax \pmod{n}$ masks the secret x with the random value $a \in \mathcal{T}_n$. Extracting x from d without knowing a is computationally infeasible, and finding x from $y = r_n[x^2]$ is as hard as factoring the modulus n.

(5b) - 1%

Does an honest client pass the login verification? Justify your answer.
Yes. The server verifies $d^2 \equiv by^c \pmod{n}$.

- If $c = 0$: $d^2 = a^2 \equiv b \pmod{n}$, which is true by the definition of b.
- If $c = 1$: $d^2 = (ax)^2 = a^2x^2 \equiv by \pmod{n}$. Since $b = r_n[a^2]$ and $y = r_n[x^2]$, the identity holds.

(5c) - 5%

Can a malicious client pass the login verification? Justify your answer.
A malicious client (who does not know x) can only pass with a very small, negligible probability. A malicious client who does not know x must guess the challenge c before committing to b. The probability of guessing correctly for one round is $1/2$. For the protocol to be successful, the challenge is repeated $t = 100$ times. The total probability of a malicious client passing is $(1/2)^{100}$, which is so low that the event is considered impossible in practice.

(5d) - 15%

We implemented a user authentication protocol as shown in Figures 1 and 3 of the assignment, using a security parameter of $t = 100$ challenge-response rounds and a 4096-bit modulus n. The implementation follows a client-server architecture using `socket` and `secrets` libraries to ensure cryptographically secure random sampling.

Registration:
First the server generates a 4096-bit random integer n and transmits it to the client. The client samples a secret $x \leftarrow \mathbb{Z}_n$ and computes the public identity $y = r_n[x^2]$, which is then sent and stored in the server's database.

Login:
First we check in the client if parameter x is `None`, which means that there hasn't been any registration yet. In this case, a message is printed to the client to register first. If x has a value, the client sends y to the server. If y does not exist in the server's database, a "Fail" message is sent,otherwise the server sends t, meaning the number of challenge rounds. For each round, the client samples a fresh $a \leftarrow \mathcal{T}_n$ such that $(a, n) = 1$, computes the value $b = r_n[a^2]$ and sends it. Then, the server provides a random challenge $c \leftarrow \mathbb{Z}_2$, and the client computes and sends $d = r_n[ax^c]$, which will either be just $a\%n$ if c is 0 or $(a * x)\%n$ if c is 1. Finally, the server grants access only if the identity $d^2 \equiv b \cdot y^c \pmod{n}$ holds for all t iterations.

(5e) - 0%

The client was modified to log the full transcript of the 4096-bit exchange (n, y, b, c, d) into a JSON formatted file, `intercepted_data.json`. We implemented a hacker script that successfully parses this data to basically make a template for the following attacks.

(5f) - 5%

Here, we used our implementation from Task 5.e changing only $t = 3$ in theserver and the hacker.

The security of this protocol against impersonation relies on the prover's inability to predict the challenge c before sending b. A malicious client who does not know x can pass a single round with probability $P = 1/2$ by guessing c. For $t = 3$, the success probability becomes $(1/2)^3 = 1/8$, or 12.5%, so "brute forcing" is actually the best way of gaining unauthorized access.
The implementation basically targets a known y and attempts login by always assuming $c = 0$, sending $b = a^2$ and $d = a$. Statistically, the hacker achieves unauthorized access within approximately 8 repeated connection attempts.

(5g) - 10%

Here, we analyzed a critical vulnerability where the client samples parameter a just once at the beginning of the login process and reuses the same parameter a for all t rounds of a login session. Reusing a leaks the secret x algebraically.
By intercepting a round where $c = 0$, the attacker obtains $d_0 = r_n[ax^0] = a \pmod{n}$. By intercepting a round where $c = 1$, the attacker obtains $d_1 = r_n[ax^1] = r_n[ax]$. Because $a \in \mathcal{T}_n$, the condition $(a, n) = 1$ is guaranteed. According to Theorem 1.31 and Corollary 2.25, the linear congruence:

$$a \cdot x \equiv d_1 \pmod{n}$$

has a unique solution for x in $\mathbb{Z}_n$. The attacker recovers x by calculating $x = r_n[d_1 \cdot a^{-1}]$ using the function `solve_linear_congruence`. Then,since they know x, they can login as the honest client, therefore gain unauthorized access.

(5h) - 2%

Now, we assume that a programming mistake happened in the server login process and the c is fixed, instead of randomly sampled.
We use our implementation from 5e, changing the server (to $c = 0$) and the hacker.
Therefore, the computations during the challenges simplify importantly: $a \in \mathbb{T}_n$ remains the same, and so does $b = r_n[a^2]$, and the difference lies in $c = 0$, that makes $d = r_n[ax^c] = r_n[ax^0] = r_n[a]$.
The check on the server side becomes $d^2 \equiv by^c \equiv b$ (mod n).

The reason why the protocol breaks is:

- $d = r_n[a],\ b = r_n[a^2]$. This makes $d^2 \equiv (r_n[a])^2 \equiv a^2$ (mod n).
- Since $b = r_n[a^2]$, we have $b \equiv a^2$ (mod n)
- From the above two, we know $d^2 \equiv a^2 \equiv b$ (mod n)

Therefore, the check always succeeds, without even needing x. This is how the attacker can gain access to a user's account, having already stolen their y from the previous tasks.
In the code, once our hacker has the client's y, presents it to the server, and the t rounds of challenges start, the hacker simply chooses an $a \in \mathbb{T}_n$, $b = a^2 \pmod{n}$, and the rest hold as mentioned above.

(5i) - 10%

Same as 5h, but for $c = 1$.
We use our implementation from 5e once again, changing the server (to $c = 1$) and the hacker.
The computations for the challenges now become: $a \in \mathbb{T}_n$ and $b = r_n[a^2]$ same as always, whereas $c = 1$, makes $d = r_n[ax^c] = r_n[ax^1] = r_n[ax]$.
The check on the server side becomes $d^2 \equiv by^c \equiv by \pmod{n}$.

The protocol breaks, since for a random round j (out of t total rounds) :

- Client samples an $a_j \in \mathbb{T}_n$, calculates $b_j = r_n[a^2]$, sends it to server
- Client receives $c_j = 1$, which is ALWAYS the case since the server will always send $c = 1$ for our current subtask
- Client then calculates $d = r_n[ax]$, sends it to the server, which then verifies $d^2 \equiv by \pmod{n}$

This is what the legitimate client does, and the malicious attacker intercepts this data.
Therefore, when the client is done, the attacker posesses all t interactions with the server, and every single one of them is legitimate, if the client actually logged in successfully.
In more detail, for every single round of challenges, the attacker posesses valid (b_j, d_j) pairs sent by the client to the server, which succeeded (with $c = 1$).
The attacker needs to do nothing more than simply use one of those pairs, and replay them to the server for all t rounds of challenges. There is absolutely no unpredictability for the c, and the server doesn't check if a pair has been reused in a previous round (at least it wasn't asked in our prompt), so this strategy should (and will) succeed.
In the code implementation, the hacker keeps the replayed b and c in two separate variables, having made sure that this pair has had $c = 1$. For all challenges that follow, `replayed_b` and `replayed_c` are encoded on the client side and sent to the server, which decodes them and accepts them every round.

(5j) - 3%

For our last subtask, we are required to modify the server and client from (5d) to use username and password.
The client no longer holds x, y. At registration and login, the client is prompted to enter a username and password.
The secret x is calculated as follows, using the password that the client will provide: Since $4096/512 = 8$, we concatenate 8 hash values. For $i = 0, 1, ..., 7$, we concatenate SHA512(i||password), i.e.

$$SHA512(0||password)||SHA512(1||password)||...||SHA512(7||password)$$

where || denotes byte concatenation. Therefore, each i is converted to 4 bytes, concatenated with the password bytes and hashed. Once all the concatenations are done, the result is converted to an integer. This is our x, and this is exactly the functionality that the helper function `get_x_from_pwd(password,n)` implements. In the end, the x is returned (mod n).
As we know, the $y = r_n[x^2]$, which is why the client doesn't need to store x or y.

In order for the client to send both y and the username to the server in one `sendall()` call, and avoid needing one more `sendall()` on the client side, and one more `recv()` on the server side, our server and client reach an agreement:
The client sends the pair(y, username), encoded and delimited by "||", as: `username.encode() + b"||" + y.to_bytes(512, "big")`, and then the server receives them and decodes them in the order they were sent.
If the server receives a pair (y, username) attempting to log in, it checks whether its database contains a registration with this username, containing this exact y. If not, it rejects it.
Finally, we would like to point out that we had some unexpected problems with the TCP sockets, that we did not encounter in previous client-server implementations, where status messages and the prompt following right after, were being mixed up in one single `recv()` call on the client, and the program froze. To counter this, for this subtask only, we created the `recv_line()` helper function, to be used instead of the original `recv()` function, ONLY where the server sends text strings (such as status confirmations "Again", "Success", "Failed", etc.). Such messages on the server side were altered to end with a newline character, and this is exactly what the helper function `recv_line()` does. It reads encoded data, until it hits a newline. At that point, it realizes that it has reached a new `sendall()` call from the server, so it decodes and returns everything it read up to the newline, and redoes this procedure for the following text after the newline.